

Course 1 Notes: Supervised Machine Learning: Regression and Classification

Adrew Ng — Jesse Hernandez

DeepLearningAI/Stanford Online — 2025

My notes for the first course in a three part machine learning specialization course consisting of three learning modules.

1. Week 1: Introduction to Machine Learning

1.1 Overview

- Key objectives

- Supervised vs unsupervised learning
- Linear regression
- Gradient Decent

- Background context

- Machine learning is poppping up in all aspects of life,e.g., google searches, streaming recommendations etc

Definition

Machine Learning

The field of study that gives computers the ability to learn without being explicitly programmed. —Arthur Samuel (1959)

1.2 Supervised Learning

Definition

Supervised Learning— Machine maps labeled inputs to outputs $x \rightarrow y$ by learning on a subset of inputs (x) with the 'correct answers' (y) to reasonably evaluate/predict outputs from new inputs.

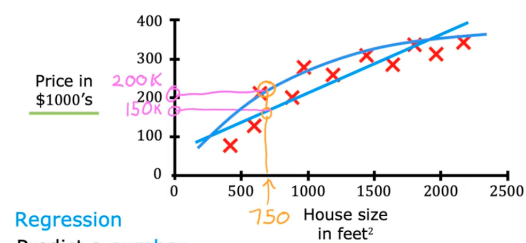
Input (X)	Output (Y)	Application
email	spam? (0/1)	spam filtering
audio	text transcripts	speech recognition
English	Spanish	machine translation
ad, user info	click? (0/1)	online advertising
image, radar info	position of other cars	self-driving car
image of phone	defect? (0/1)	visual inspection

Example

Regression— Predicting a number with a continuous set of outputs.

- Fit some input data to evaluate the house price using the best fit
- Learn about ML algorithms to pick the best evaluating tool, e.g., linear or quadratic model

Regression: Housing price prediction

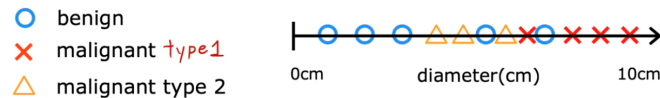


Example

Classification— Predicting a value from a finite set of possible outputs, e.g., benign(0) or malignant(1) tumor based on some input data (tumor size)

- Class or category used interchangeably for output value
- Can have multiple inputs (right image) where the ML algorithm forms a boundary on multi-dimensional data to predict the classification

Classification: Breast cancer detection



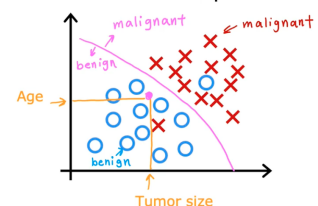
class category
 Classification
 predict categories cat dog benign malignant 0, 1, 2
 small number of possible outputs

Stanford ONLINE

DeepLearning.AI

Andrew Ng

Two or more inputs



1.3 Unsupervised Learning

Definition

Unsupervised Learning— Find patterns or structures in unlabeled data

- *Clustering*: group unlabeled data
- *Anomaly detection*: detect ambiguous or outliers in data
- *Dimensionality reduction*: compress large data sets into smaller ones without losing the important information

Example

Clustering— groups data into clusters that have similar characteristics

- google news example clusters articles by similar keywords in article titles to group similar stories
- dna grouping for types of people
- grouping customers with similar goals to predict best suggestions



1.4 Linear Regression

Definition

Training Set: data used to train the model consisting of m input (x^i) and output (y^i) variables with (x^i, y^i) as the i th training point.

Learning Algorithm: Takes the training set and outputs a function to predict the output ($\hat{y}$) from the input feature, (x), e.g.,

$$x \rightarrow \boxed{f} \rightarrow \hat{y}$$

Note

For supervised learning the training set contains the ‘correct output’ (y), not to be confused with the estimated output from the model ($\hat{y}$)

- Most widely used ML model in real world applications, e.g.,
- For linear regression the function or model, f , is a linear function:

$$f(x) = wx + b \tag{1}$$

where w , b are parameters of function.

- Foundational model for estimating outputs, also called **univariate** linear regression (one variable) and basis for more complex models, e.g., quadratic.

1.5 Cost Function

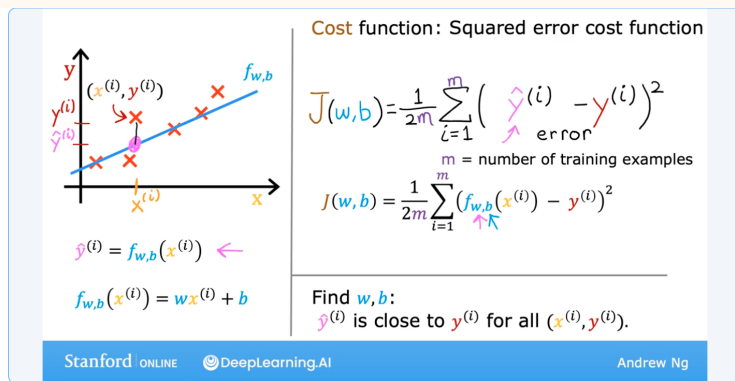
Definition

Cost Function tells us how good the model fits to the data and can be expressed in many forms, see example of squared error cost function $J(w, b)$, and can be minimized to find the best parameters for the model.

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m \left(f_{w,b}(x^{(i)}) - y^{(i)} \right)^2 \quad (2)$$

Example

- Squared error cost function compares the predicted value ($\hat{y}$) from the linear model $f(x) = w \cdot x + b$ to the true value (y).

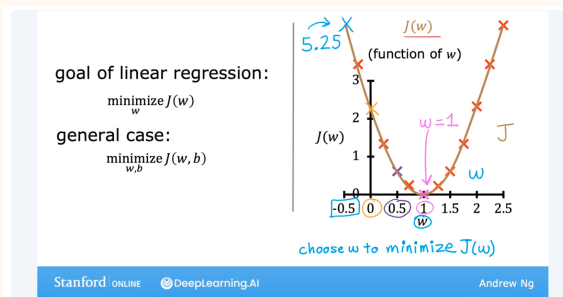
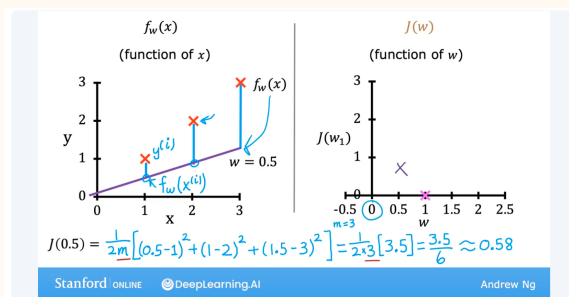


1.5.1 Parameter Estimation with Cost Function

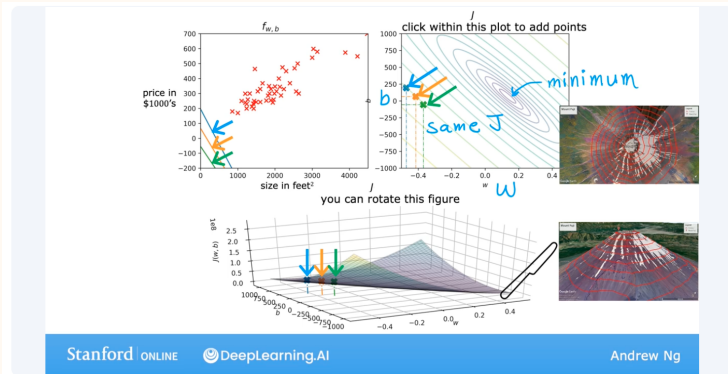
Example

- By minimizing the Cost Function we can get best parameters that fit our data
- The example below used simplified linear model with no bias ($b = 0$) and the squared error cost function
- We see the parameter that minimized J (smallest value) $J(1)$ gives the best fit to the data.

$$J(w) = \frac{1}{2m} \sum_{i=1}^m (f_w(x^{(i)}) - y^{(i)})^2 = \frac{1}{2m} \sum_{i=1}^m (wx^{(i)} - y^{(i)})^2 \quad (3)$$



- For the complex function with multiple parameters (2 in this example) the cost function is a 3D plot and can be visualized as a contour plot to estimate many parameters.



1.6 Gradient Decent

Definition

Gradient Decent is an algorithm used to minimize any functions including the cost function discussed by taking steps towards the greatest decent of the function.

$$\begin{aligned} w &:= w - \alpha \frac{\partial}{\partial w} J(w, b) \\ b &:= b - \alpha \frac{\partial}{\partial b} J(w, b) \end{aligned} \quad (4)$$

where α is the ‘learning rate’ that dictates the size of the steps and the equations above are updated simultaneously for two-parameter cost function by reassigning the parameter value at each step.

Note

The **Gradient Term** in the above set of equations determines whether the parameter is decreasing or increased based on the slope of the cost function $\frac{\partial}{\partial w} J(w, b, ..)$ for all parameters until the parameters reach a local/global minimum.

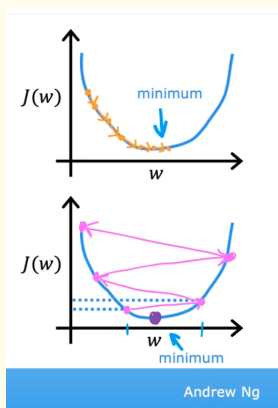
$$\frac{\partial}{\partial w} J(w, b, ..) > 0 \quad \therefore \quad w_{new} < w \quad (5)$$

$$\frac{\partial}{\partial w} J(w, b, ..) < 0 \quad \therefore \quad w_{new} > w \quad (6)$$

Note

Learning rate must be chosen to optimize speed of algorithm and likelihood of convergence:

How learning rate affects minimization:



- If the learning rate is too small the algorithm is extremely slow and inefficient. (Top image with tiny steps)
- If the learning rate is too large then the algorithm could fail by missing the minimum or fail to converge. (Bottom image with massive steps)
- Near a local minimum the slope(rate of descent) becomes smaller and thus the parameters change by smaller values so with a fixed α the ‘steps’ become smaller

1.6.1 Implementation

- Correct implementation of simultaneously updating parameters (skeleton code)

- The updating of the parameters must be done after the determination of the new parameters using the previous ones first.

Correct: Simultaneous update	Incorrect
$\begin{aligned} \text{tmp_w} &= w - \alpha \frac{\partial}{\partial w} J(w, b) \\ \text{tmp_b} &= b - \alpha \frac{\partial}{\partial b} J(w, b) \\ w &= \text{tmp_w} \\ b &= \text{tmp_b} \end{aligned}$	$\begin{aligned} \text{tmp_w} &= w - \alpha \frac{\partial}{\partial w} J(w, b) \\ w &= \text{tmp_w} \\ \text{tmp_b} &= b - \alpha \frac{\partial}{\partial b} J(w, b) \\ b &= \text{tmp_b} \end{aligned}$

Example

Lets apply gradient decent (??) to the linear regression model (??) parameters using the squared errors cost function (??). The gradient decent algorithm then decomposes to multiple iterations of the solutions of the parameters with some initial guess (w, b) :

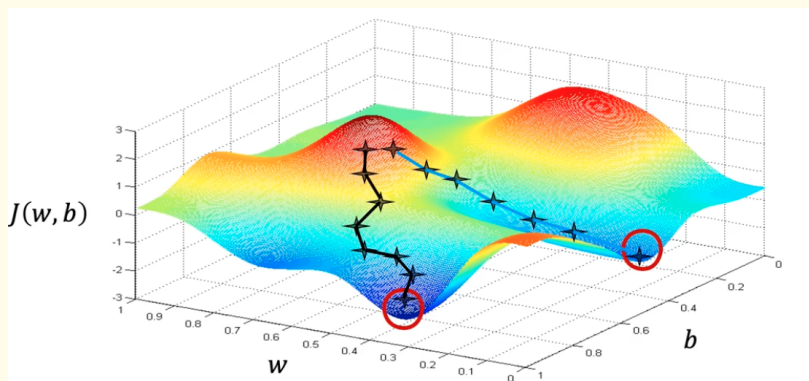
$$w' = w - \alpha \left[\frac{1}{m} \sum_{i=1}^m \left(f_{w,b}(x^{(i)}) - y^{(i)} \right) x^{(i)} \right] \quad (7)$$

$$b' = b - \alpha \left[\frac{1}{m} \sum_{i=1}^m \left(f_{w,b}(x^{(i)}) - y^{(i)} \right) \right] \quad (8)$$

where the bracket terms have been evaluated and are the partial derivatives of the cost function with the linear regression model.

Note

- When solving for more complex cost functions, the gradient decent algorithm can lead to different paths and get 'stuck' in local minima, depicted below:



- The special case of the squared errors cost function take a bowl shape and only has a single global minimum.

2. Week 2: Regression with multiple input variables

2.1 Overview

- Key objectives

- Vectorization
- Gradient decent for multiple linear regression (many features, x_i)
- Feature Scaling
- Feature Engineering
- Polynomial Regression

- Implementation

- Checking gradient decent convergence
- choosing learning rate
- Using scikit-learn for gradient descent and regression models

Note

Multiple Features or inputs to your training sets are defined as $x_j^{(i)}$, where their can be n features/input values for i^{th} training sample.

Multiple Linear Regression is then a linear regression model with multiple features and can be written in terms of vectors

$$f_{\vec{w},b} = \vec{w} \cdot \vec{x} + b, \quad (9)$$

where the vector is a list of values defined in the normal way $\vec{v} = (v_1, v_2, \dots, v_n)$ and the dot product is defined as the sum of the product of each vector element-wise

$$\vec{a} \cdot \vec{b} = \sum_{i=0}^{n-1} a_i b_i \quad (10)$$

2.2 Vectorization

- Make code shorter and more efficient
- Take advantage of modern linear algebra libraries and gpu hardware using parallel processing to scale to large data sets
- Use arrays in NumPy python library, e.g., `w = np.array([1.0, 2.5, -3.3])`

Note

Broadcasting is how numpy treats arrays of different sizes by means of vectorizing array operations in C instead of python (documentation)

Example

– Implementation of vectors for processing where the dot product is standardized with built-in NumPy methods.

Parameters and features

$\vec{w} = [w_1 \ w_2 \ w_3] \quad n=3$

b is a number

$\vec{x} = [x_1 \ x_2 \ x_3]$

linear algebra: count from 1

NumPy

```
w = np.array([1.0, 2.5, -3.3])
b = 4
x = np.array([10, 20, 30])
```

code: count from 0

Without vectorization $n=100,000$

$f_{\vec{w},b}(\vec{x}) = w_1x_1 + w_2x_2 + w_3x_3 + b$

```
f = w[0] * x[0] +
    w[1] * x[1] +
    w[2] * x[2] + b
```

☹️

Without vectorization

$$f_{\vec{w},b}(\vec{x}) = \left(\sum_{j=1}^n w_j x_j \right) + b$$

$\sum_{j=1}^n \rightarrow j=1 \dots n$
 $1, 2, 3$

```
f = 0
for j in range(0, n):
    f = f + w[j] * x[j]
f = f + b
```

☹️

Vectorization

$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$

```
f = np.dot(w, x) + b
```

Stanford ONLINE DeepLearning.AI Andrew Ng

– Implementation of vectorization in gradient descent for 16 parameter linear regression

Gradient descent

$\vec{w} = (w_1 \ w_2 \ \dots \ w_{16})$ ~~b~~ parameters

derivatives $\vec{d} = (d_1 \ d_2 \ \dots \ d_{16})$

```
w = np.array([0.5, 1.3, ... 3.4])
d = np.array([0.3, 0.2, ... 0.4])
```

compute $w_j = w_j - 0.1d_j$ for $j = 1 \dots 16$

Without vectorization

```
w1 = w1 - 0.1d1
w2 = w2 - 0.1d2
...
w16 = w16 - 0.1d16
```

```
for j in range(0, 16):
    w[j] = w[j] - 0.1 * d[j]
```

With vectorization

$\vec{w} = \vec{w} - 0.1\vec{d}$

$0.1 * \vec{d}$

$\vec{w} = \vec{w} - 0.1 * \vec{d}$

Stanford ONLINE DeepLearning.AI Andrew Ng

2.2.1 Gradient Decent with vectorization

Definition

– vector notation

Parameters $\vec{w} = [w_1 \dots w_n$

Model $f_{\vec{w},b} = \vec{w} \cdot \vec{x} + b$

Cost Function $J(\vec{w}, b)$

$$\textbf{Gradient Decent} \quad \text{repeat} \begin{cases} w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b) \\ b = b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b) \end{cases} \quad (11)$$

– Gradient decent with square errors cost function for multiple linear regression then becomes

$$\text{repeat} \begin{cases} \vec{w} = \vec{w} - \alpha \frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) \vec{x}^{(i)} \\ b = b - \alpha \frac{\partial}{\partial b} \frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) \end{cases}, \quad (12)$$

where each component of feature w_j is compactly written as a vector $\vec{w}$ and each feature is simultaneously updated along with b .

Note

Normal Equation can be used for linear regression as an alternative to gradient decent where linear algebra libraries can solve for the best parameters in one go instead of through many iterations.

–Disadvantages:

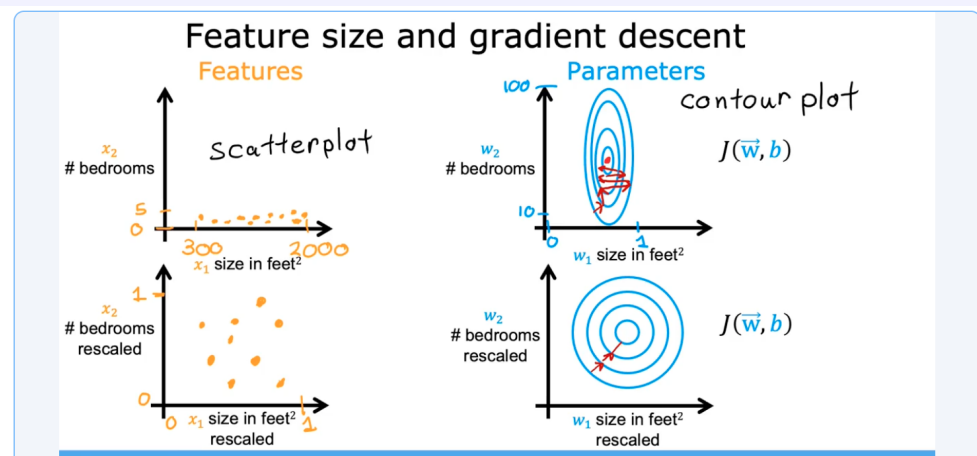
- Only useful for linear regression and does not generalize to other learning algorithms
- Slow for large number of features ($>10k$)

2.3 Feature Scaling

Definition

Feature Scaling is an algorithm that allows for the scaling of features of your model to speed up gradient decent.

- scaling of parameters typically proportional to range of feature, e.g., a feature that spans $(0, 5)$ will have a larger scaling than a feature ranging $1000 - 5000$ (intuitive)
- The feature scaling allows for the values of your equations to have similar scaled values and is important for the gradient decent algorithm spacing to run smoothly, e.g., the phase space of parameters with vastly different ranges is difficult to analyze efficiently (shown by the contour plots below)



Example

- **Maximum normalization** scales features in your model by the maximum value of the range to force them to be in smaller range, e.g., $(min, 1)$:

$$x_{i, scaled} = \frac{x_i}{\max\{x_i\}} \quad (13)$$

where $\max$ is just the maximum value of the feature.

- **Mean normalization** scales the features by the mean of the feature (μ_i) :

$$x_{i, scaled} = \frac{x_i - \mu_i}{R_{x_i}} \quad (14)$$

where the range of the feature is labeled $R_{x_i} \equiv \max\{x_i\} - \min\{x_i\}$.

This choice of scaling will range from $(-1, 1)$ and is a symmetric range.

- **Z-score normalization** scales the feature by using the standard deviation σ_i of the feature:

$$x_{i, scaled} = \frac{x_i - \mu_i}{\sigma_i} \quad (15)$$

where the standard deviation gives us a measure of the spread of the data in statistics.

2.4 Convergence of gradient decent

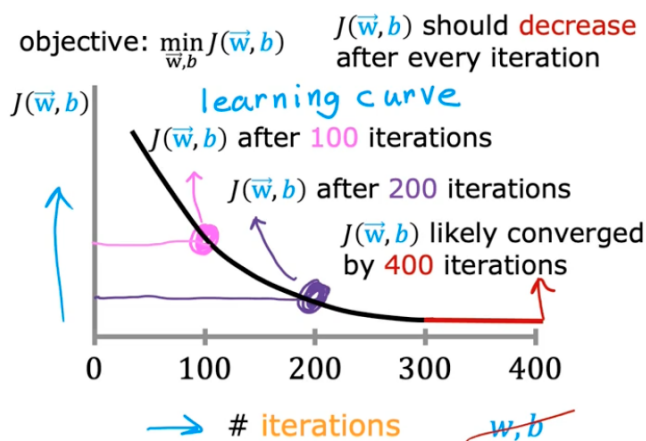
Definition

Learning Curves give an indicator of how the algorithm is working over the evolution of iterations in the case of minimizing the cost function (example below).

- Can take many forms for different applications in ML

Example

- Plot a learning curve where the cost function is plotted against the # of iterations (below)
- The learning curve should be a smooth descending function, converging to singular value.
 - If not than its a sign that the learning rate α may be too large (example where we pass the minima)
- Also give a feel for the number of iterations needed to converge

**Note**

- Can also rely on automatic convergence tests where:

Let ϵ be a small number, e.g., 10^{-3}

If $J(\vec{w}, b)$ decreases by $\leq \epsilon$ in one iteration than we declare convergence

- Its usually still useful to illustrate the convergence with learning curves

2.4.1 Choosing Learning Rate**Note**

- By looking at the leaning curve we can verify that the cost function is smoothly decreasing and the learning rate is not too large.
- By looking at the rate of the decent for convergence of the cost function we can adjust the learning rate to be more efficient without being too small.
- If the learning curve is always increasing than we may have a bug in our code

2.5 Feature Engineering

Definition

Feature engineering is the practice of designing new features by transforming or combining original features that could be better predictors for the model, e.g., for house prices given the frontage (x_1) and depth (x_2) of the lot the area ($x_3 = x_1x_2$) can be defined and used in the model, where

$$f_{\vec{w},b}(\vec{x}) = w_1x_1 + w_2x_2 + w_3x_3 + b$$

to improve the model.

2.6 Polynomial regression

Definition

Polynomial regression is a model to fit non-linear data such as polynomial functions of different orders, e.g., a polynomial regression to n order,

$$f_{\vec{w},b}(\vec{x}) = w_1x + w_2x^2 + w_3x^3 + \cdots + w_nx^n + b \quad (16)$$

where a cut off can be made to best fit the data.

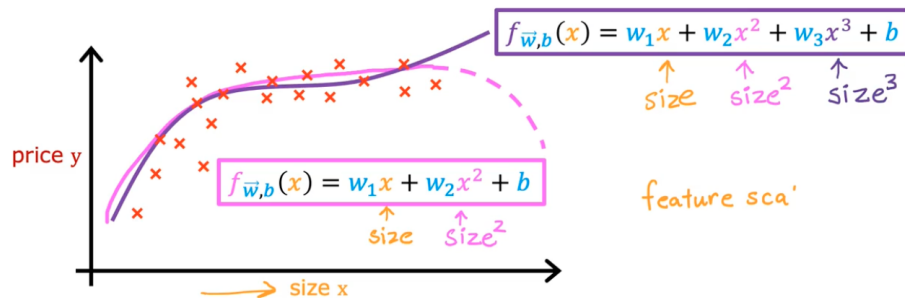
- Have to be particularly careful to use feature scaling because now the size of each term grows quadratic in the feature.

Example

Choice of features to use when feature engineering using polynomial regression:

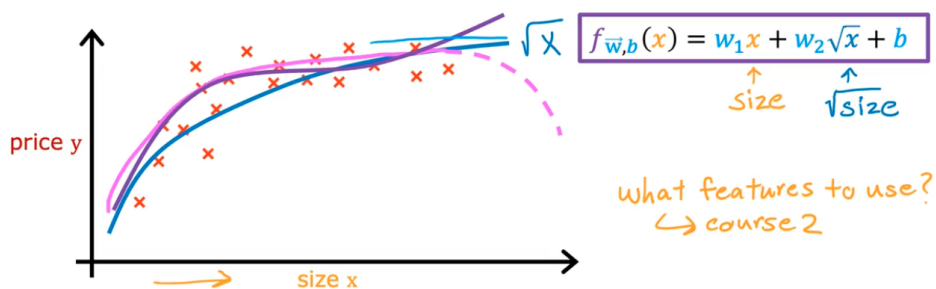
- polynomial to the second (x^2) or third power (x^3) for a feature, e.g., price vs house sizes below

Polynomial regression



- polynomial with power 1/2 ($\sqrt{x}$) for a monotonically increasing fit

Choice of features



3. Week 3: Classification

3.1 Overview

- Key objectives

- Classification
- Logistic regression
- Decision Boundary
- Overfitting
- Regularization

- Implementation

- Sigmoid function
- Cost function for logistic regression
- Gradient descent for logistic regression
- Regularized linear regression
- scikit-learn implementation

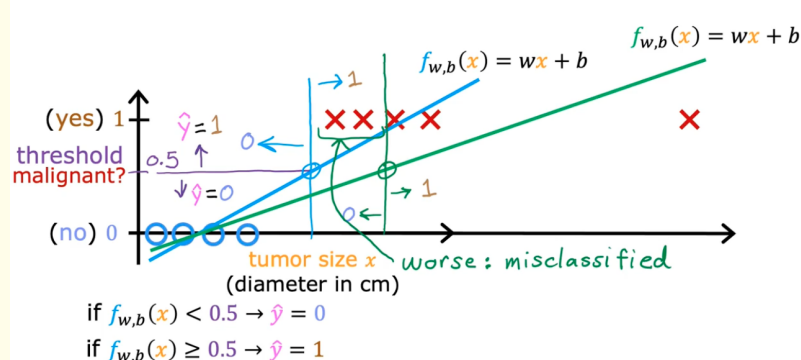
Definition

Binary classification have two potential outputs, e.g., (yes,no), (true,false(0))

- ‘Negative’ class (false) vs ‘Positive’ class (true) where class/category is interchangeable for outputs

Note

Linear regression is no longer a good model for classification where we see how fitting a binary data set leads to misrepresentation of the predictions below, e.g.,



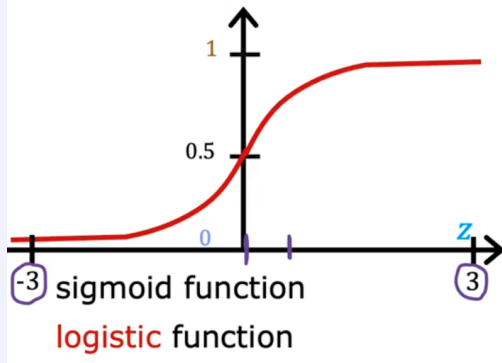
3.2 Logistic Regression

Definition

Sigmoid/logistic function is a function that gives an output of 0 or 1 and is valuable for modeling two-level systems represented by:

$$g(z) = \frac{1}{1 + e^{-z}} \quad 0 < g(z) < 1. \quad (17)$$

A visual representation of the function is:



Note

Logistic Regression model is written in terms of the sigmoid function (??) where our independent variable z is represented by a linear function of our feature $\vec{x}$:

$$z = \vec{w} \cdot \vec{x} + b. \quad (18)$$

This transformation is used to construct the logistic regression model:

$$f_{\vec{w},b}(\vec{x}) = g(\vec{w} \cdot \vec{x} + b) = \frac{1}{1 + \exp[-(\vec{w} \cdot \vec{x} + b)]} \quad (19)$$

- The linear regression algorithm outputs a value from (0,1) and thus acts like a probability to determine the classification and can be written technically as:

$$f_{\vec{w},b}(\vec{x}) = P(y = 1 | \vec{x}; \vec{w}, b) \quad (20)$$

that says what is the probability of $y = 1$ for the feature $\vec{x}$ given the values of $\vec{w}, b$.

3.2.1 Decision Boundary

Definition

The **Decision Boundary** is the threshold of the model to predict a binary result either true or false and can be defined when the transformed variable $z = 0$, e.g., for the linear regression this would be $z = \vec{w} \cdot \vec{x} + b = 0$ and represents when the decision is not quit 0 or 1.

- Due to the continuous nature of the sigmoid function where recall the outputs can take any value (0,1) interpreted as a probability.
- The typical threshold is for the model is ≤ 0.5 whether the results are predicted true or false

Example

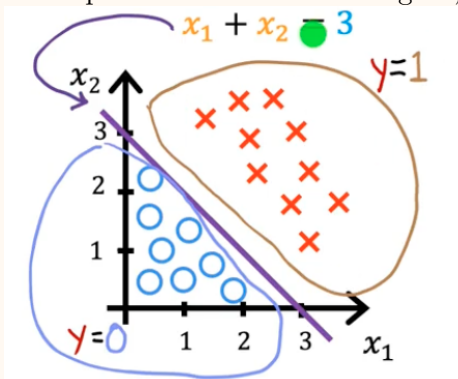
We look at a simple example with two features and transform our **multiple linear regression** to logistic regression using ??, so now the function takes the form:

$$f_{\vec{w},b}(\vec{x}) = g(w_1x_1 + w_2x_2 + b). \quad (21)$$

To find the decision boundary we solve for $z = 0$ with parameters $\vec{w} = (1,1)$, $b = -3$) for the above model to get the boundary ??:

$$\begin{aligned} z &= x_1 + x_2 - 3 = 0 \\ x_1 + x_2 &= 3 \end{aligned} \quad (22)$$

and represented in the below figure, where the boundary is depicted



Example

We can also look at a **non-linear decision boundary** with two features with the underlying model being of the form of polynomial regression taking the form:

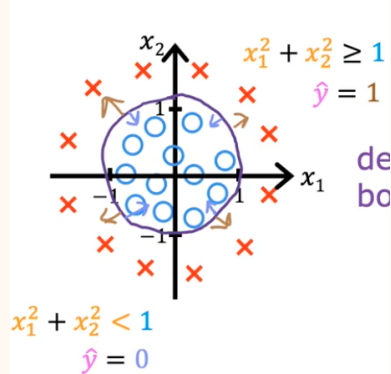
$$z = w_1 x_1^2 + w_2 x_2^2 + b. \quad (23)$$

Again solving for the decision boundary by solving $z = 0$ with parameters $\vec{w} = (1, 1)$, $b = -1$ we now get a circular boundary:

$$z = x_1^2 + x_2^2 + -1 = 0 \quad (24)$$

$$x_1^2 + x_2^2 = 1 \quad (25)$$

represented by the figure below:



3.2.2 Cost function

Note

- The squared error cost function is no longer a convex function to minimize for logistic regression model ??.
- Can write the cost function in terms of the loss function ?? and this new cost function is a convex function to find a global minima for minimizing the model parameters.
- Below the concept underpinning the equations for the cost function is **Maximum Likelihood estimation**

Definition

The **Cost function** for logistic regression can be written in terms of a loss function:

$$J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m L\left(f_{\vec{w}, b}(\vec{x}^{(i)}), y^{(i)}\right) \quad (26)$$

where the **Logistic Loss function** tells us the likelihood (probability from 0 to 1) of fitting the model well is defined as

$$L\left(f_{\vec{w}, b}(\vec{x}^{(i)}), y^{(i)}\right) = (-y^{(i)} \log(f_{\vec{w}, b}(\vec{x}^{(i)})) - (1 - y^{(i)}) \log(1 - f_{\vec{w}, b}(\vec{x}^{(i)}))) \quad (27)$$

and can be written for both cases:

$$L\left(f_{\vec{w}, b}(\vec{x}^{(i)}), y^{(i)}\right) = \begin{cases} -\log(f_{\vec{w}, b}(\vec{x}^{(i)})) & \text{if } y^{(i)} = 1 \\ -\log(1 - f_{\vec{w}, b}(\vec{x}^{(i)})) & \text{if } y^{(i)} = 0. \end{cases} \quad (28)$$

Finally the cost function can be rewritten by plugging in ?? into ??:

$$J(\vec{w}, b) = -\frac{1}{m} \sum_{i=1}^m \left[(-y^{(i)} \log(f_{\vec{w}, b}(\vec{x}^{(i)})) - (1 - y^{(i)}) \log(1 - f_{\vec{w}, b}(\vec{x}^{(i)}))) \right], \quad (29)$$

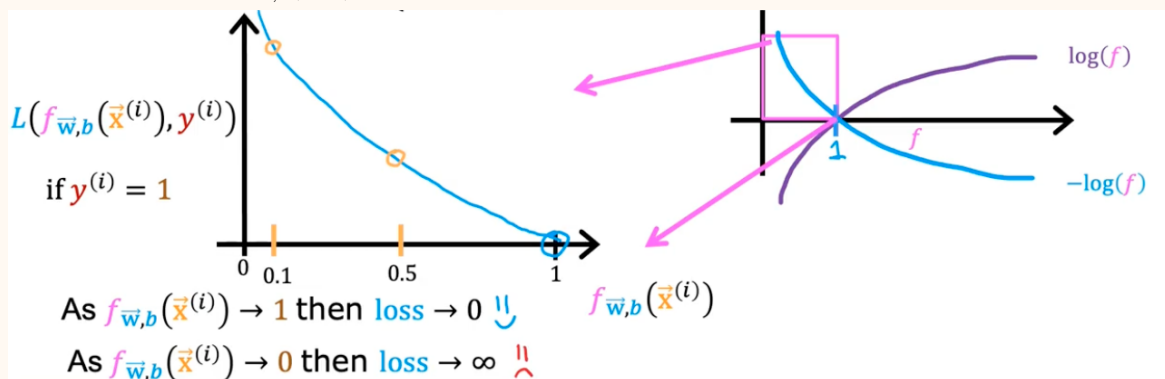
where this is the useful form for programming the loss for gradient descent.

Example

Example of cases for loss function determination:

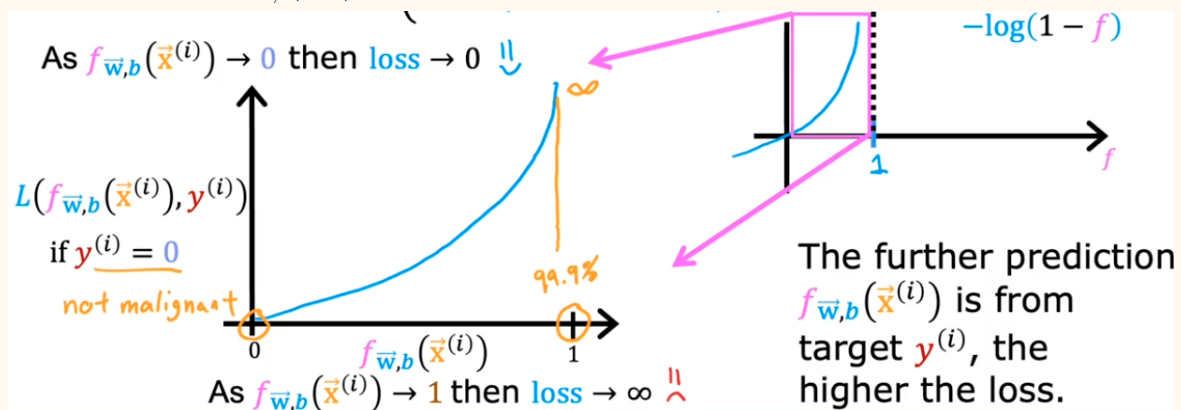
Case 1: $y^{(i)} = 1$,

- As the model $f_{\vec{w},b}(\vec{x}^{(i)}) \rightarrow 1$ the loss function $L \rightarrow 0$ and the prediction is close to the true value (L is small)
- Similarly as $f_{\vec{w},b}(\vec{x}^{(i)}) \rightarrow 0$ $L \rightarrow \infty$ and penalizes the model to find a smaller loss



Case 2: $y^{(i)} = 0$,

- As the model $f_{\vec{w},b}(\vec{x}^{(i)}) \rightarrow 0$ the loss function $L \rightarrow 0$ and the prediction is close to the true value (L is small) penalizes the model to find a smaller loss
- Similarly as $f_{\vec{w},b}(\vec{x}^{(i)}) \rightarrow 1$, $L \rightarrow \infty$ and penalizes the model to find a smaller loss



3.2.3 Gradient descent implementation

Note

The gradient decent algorithm for logistic regression can be calculated by using the definition for multiple features ?? with the logistic regression model ?? where the derivative of the cost functions using ?? is:

$$\frac{\partial}{\partial w_j} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m \left(f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)} \right) x_i^{(i)}, \quad (30)$$

$$\frac{\partial}{\partial b} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m \left(f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)} \right), \quad (31)$$

which looks similar to linear regression but $f_{\vec{w}, b}(\vec{x}^{(i)})$ is now the sigmoid function ??.

3.3 Problem of overfitting

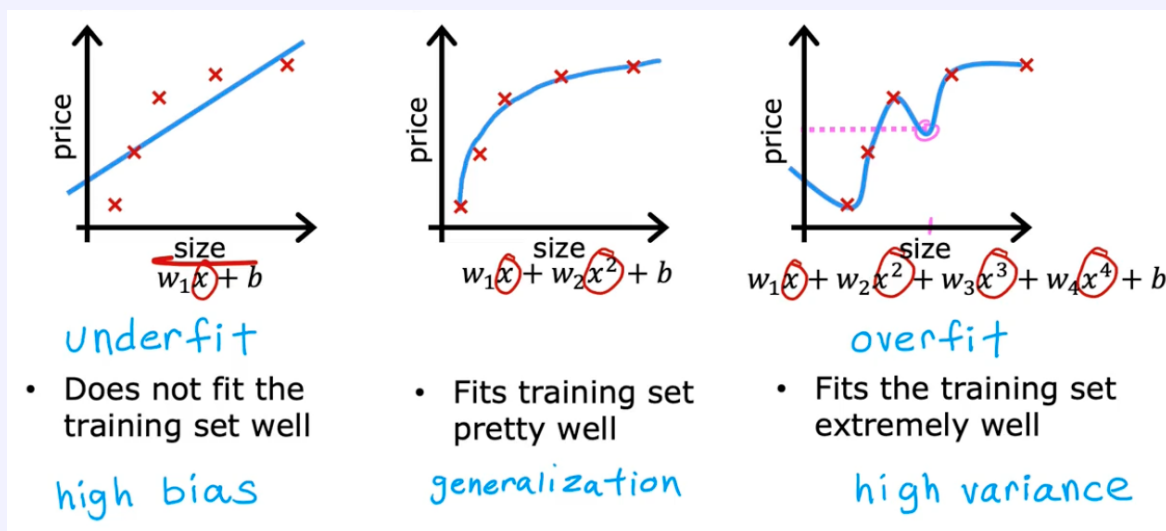
The goal of machine learning regression is to find the model that fits the data just right.

Definition

When the data is **Underfit** or has **High Bias** the model does not fit the training set well, e.g., an oversimplified model to data

When the data is **Overfit** or has **High Variance** the model fits the data too well

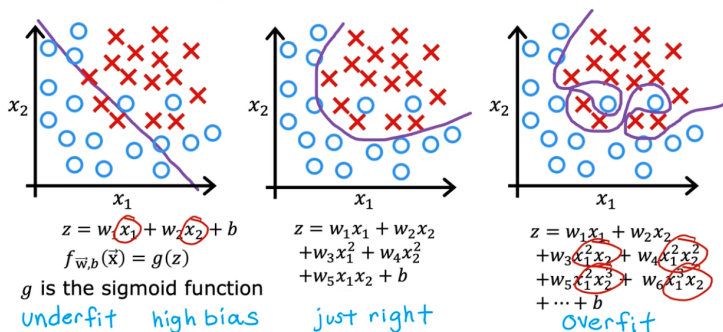
- can have instability of predictions (high variance) when training with slightly different training set (does not generalize well)
- can model many features that are unrealistic in the data, e.g., the third image below shows a dip in the housing prices at the dotted magenta line which is inconsistent with the shape of the data



Example

For this example, different logistic regression models are used to classify the boundary function for finding a malignant tumor.

Classification



3.3.1 Addressing the problem

Note

To address overfitting we can:

- collect more training examples
- reduce/change the set of features used (simplify model), e.g., **Feature Selection** (discussed in course 2)
- reduce the size of parameters, e.g., **Regularization**

3.4 Regularization

Definition

Regularization is used to reduce weights of features in our model without having to omit any features.

The regularized cost function example using the squared errors cost function (?? is modified by adding a constraining term (regularization term) introducing the **regularization parameter**, (λ):

$$J_{\text{regularized}}(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^m \left(f_w(x^{(i)}) - y^{(i)} \right)^2 + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2 + \left[\frac{\lambda}{2m} b^2 \right] \quad (32)$$

where $\lambda > 0$ and the term to regularize b may be excluded without drastically changing the result.

- The regularization term balances fitting the data well and keeping the weights w_j of the model small.
- If $\lambda = 0$ the model will not reduce the weights and overfit the model.
- if λ is too large the $w_j \rightarrow 0$ because of the big penalty when minimizing and underfit the model.

[Note] This is similar to the Lagrange multipliers method used in least squares fitting in my research

3.4.1 Regularized Regression with gradient descent

Note

The gradient decent algorithm for regularized linear regression can be written using ?? and using the definition of gradient decent (??) we just need the derivatives of the regularized cost function:

$$\frac{\partial}{\partial w_j} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m \left(f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)} \right) x_i^{(i)} + \frac{\lambda}{m} w_j, \quad (33)$$

$$\frac{\partial}{\partial b} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m \left(f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)} \right), \quad (34)$$

Deeper Dive

If we plug these derivatives back into the gradient decent (??) and factor out and isolate w_j in ?? then we can rewrite it the updated parameters as:

$$w_j^{new} = w_j \left(1 - \alpha \frac{\lambda}{m} \right) - \alpha \frac{1}{m} \sum_{i=1}^m \left(f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)} \right) x_i^{(i)} \quad (35)$$

where we can see that the regularization just shrinks the original parameter by a factor of $(\alpha \frac{\lambda}{m})$ and the second term is the normal update without regularization.

Very generally the above procedure can be used for logistic regression where the cost function is now ??.

3.5 Summary